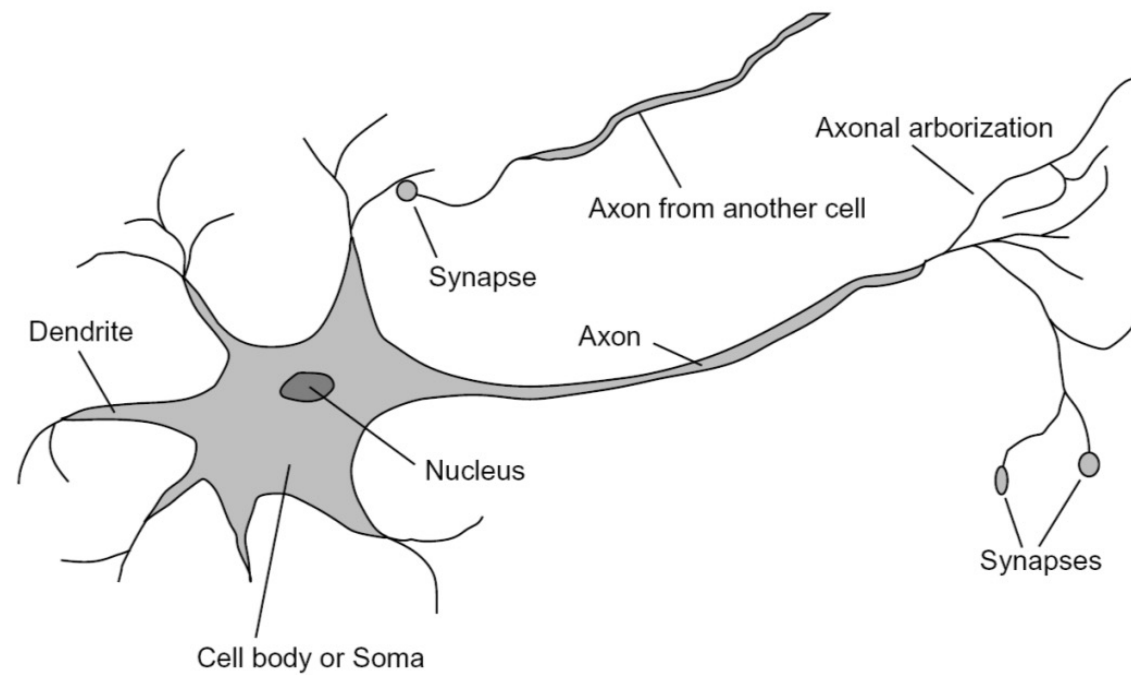
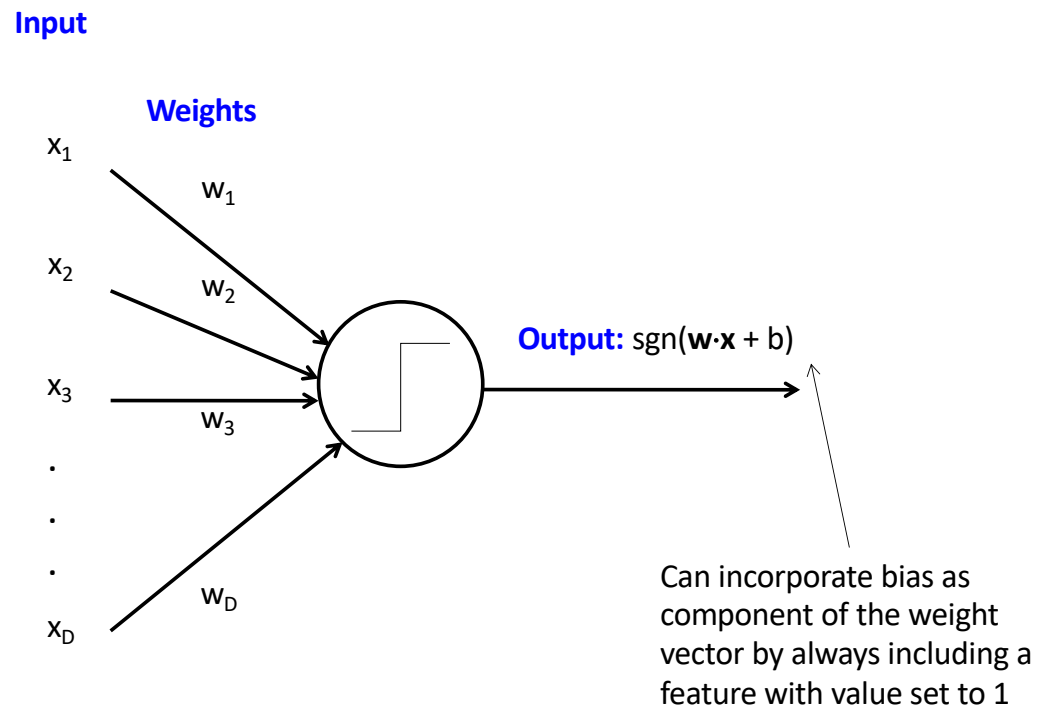


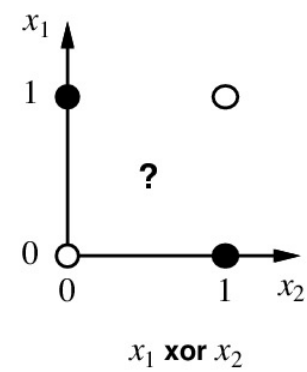
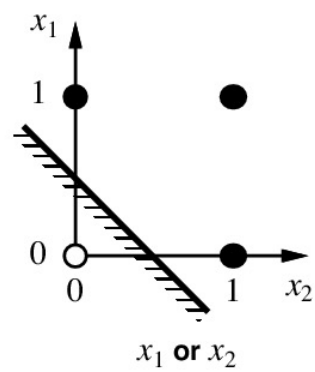
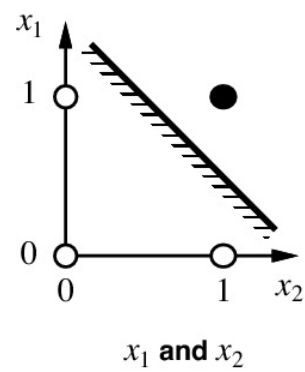
Loose inspiration: Human neurons



Perceptron

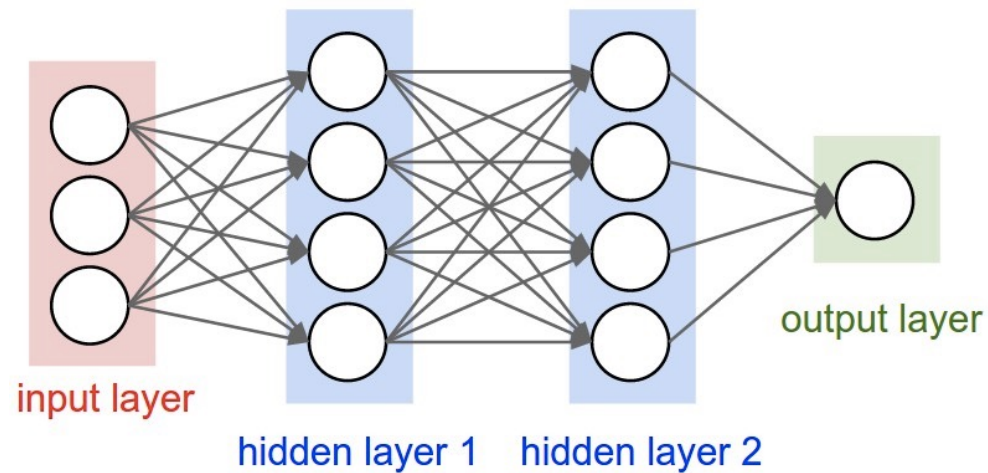


Linear separability



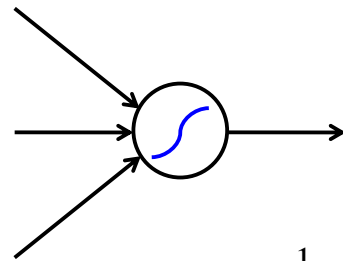
How do we make nonlinear classifiers out of perceptrons?

- Build a multi-layer neural network!

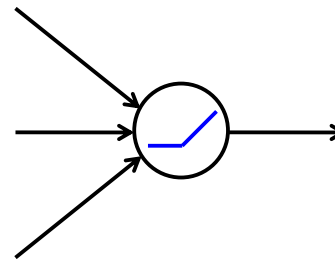


Training of multi-layer networks

- **Gradient descent** requires neural networks to be equipped with a (nearly) differentiable nonlinearity function, called **neuron**



Sigmoid: $g(t) = \frac{1}{1 + e^{-t}}$



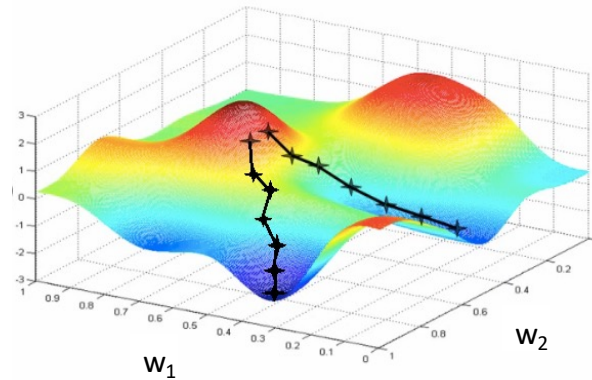
Rectified linear unit (ReLU): $g(t) = \max(0, t)$

Training of multi-layer networks

- Find network weights to minimize the error between true and estimated labels of training examples:

$$E(\mathbf{w}) = \sum_{j=1}^N (y_j - f_{\mathbf{w}}(\mathbf{x}_j))^2$$

- Update weights by **gradient descent**: $\mathbf{w} \leftarrow \mathbf{w} - \alpha \frac{\partial E}{\partial \mathbf{w}}$



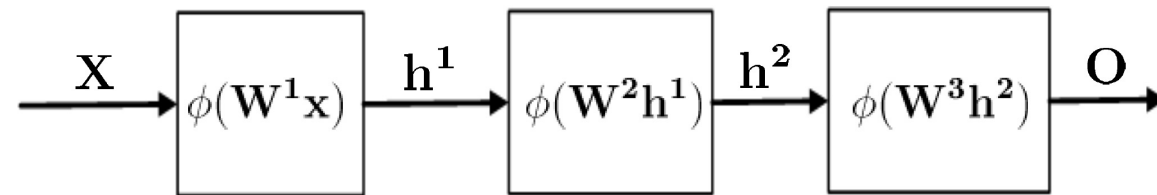
Perceptron training algorithm

- Initialize weights
- Cycle through training examples in multiple passes (*epochs*)
- For each training example:
 - Classify with current weights: $y' = \text{sgn}(\mathbf{w} \cdot \mathbf{x})$
 - If classified incorrectly, update weights:

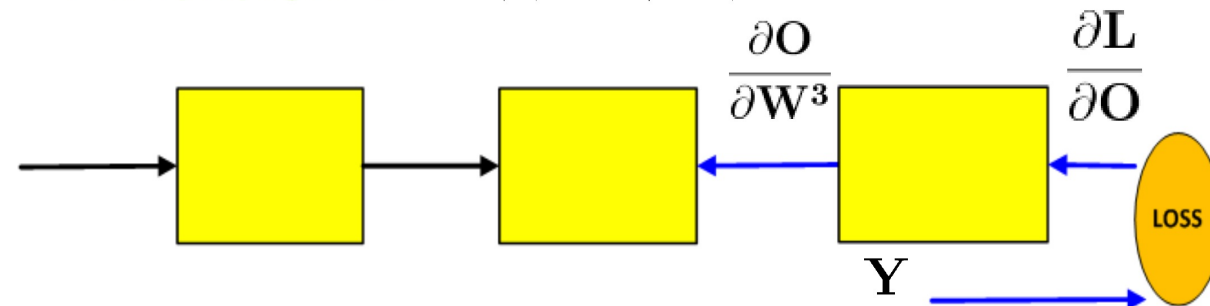
$$\mathbf{w} \leftarrow \mathbf{w} + \alpha (y - y') \mathbf{x}$$

- α is a *learning rate* that should decay as a function of epoch t , e.g., $1000/(1000+t)$

Forward-Backward Propagation



Forward propagation: $h(x) = \phi(Wx)$



Backward propagation: $\frac{\partial L}{\partial W^3} = \frac{\partial L}{\partial O} \frac{\partial O}{\partial W^3}$ (Chain Rule)

NNs are Universal Approximators (in theory)

Let $\varphi(\cdot)$ be a nonconstant, **bounded**, and **monotonically**-increasing **continuous** function. Let I_m denote the m -dimensional **unit hypercube** $[0, 1]^m$. The space of continuous functions on I_m is denoted by $C(I_m)$. Then, given any $\varepsilon > 0$ and any function $f \in C(I_m)$, there exist an integer N , real constants $v_i, b_i \in \mathbb{R}$ and real vectors $w_i \in \mathbb{R}^m$, where $i = 1, \dots, N$, such that we may define:

$$F(x) = \sum_{i=1}^N v_i \varphi(w_i^T x + b_i)$$

as an approximate realization of the function f where f is independent of φ ; that is,

$$|F(x) - f(x)| < \varepsilon$$

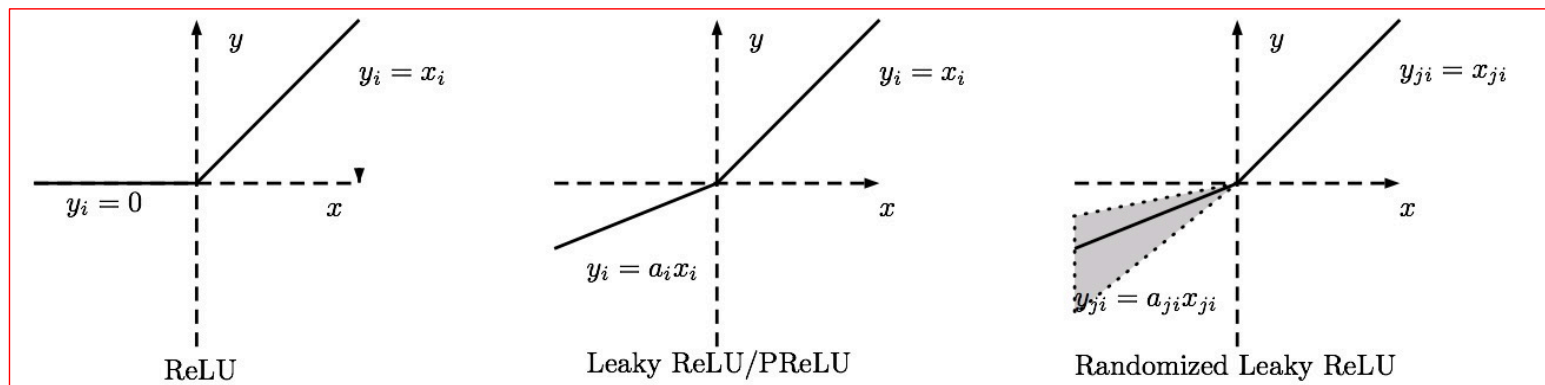
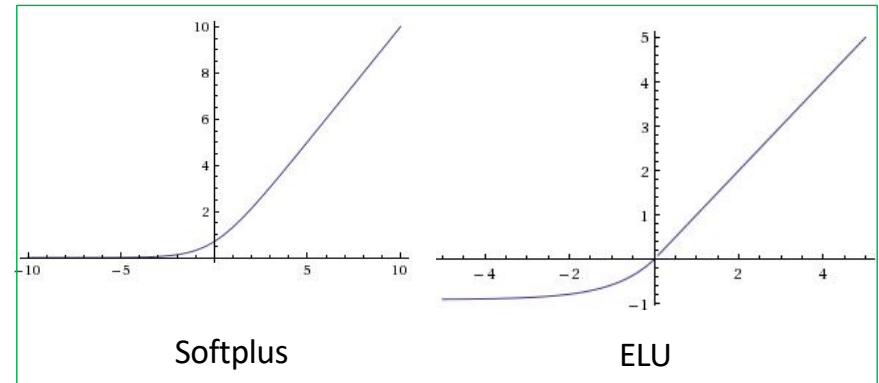
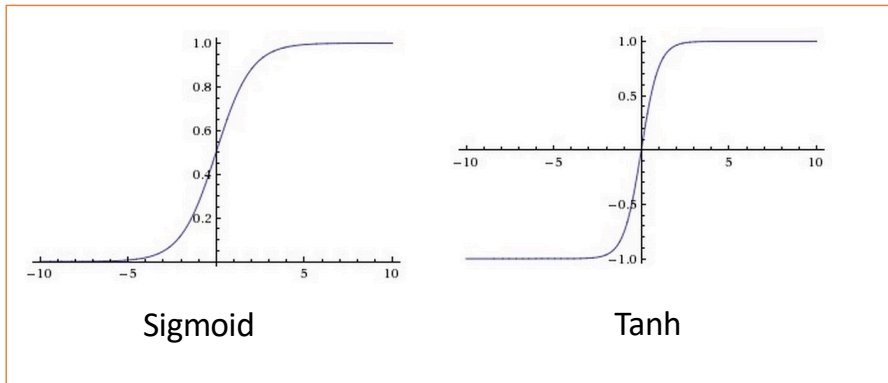
for all $x \in I_m$. In other words, functions of the form $F(x)$ are **dense** in $C(I_m)$.

- A feed-forward network with a **single hidden layer** containing a **finite number of *nonlinear*** neurons, can approximate **any continuous function on compact subsets of $\mathbb{R}^n$** , under mild assumptions.
- It is not the specific choice of the activation function, but rather the **multilayer feedforward architecture** itself which gives neural networks the potential of being universal approximators.
- It does not touch upon the **algorithmic learnability** of those parameters.

Weight Initialization

- All Zero Initialization: **Terribly Wrong!**
 - If every neuron in the network computes the same output, then they will also all compute the same gradients during back-propagation and undergo the exact same parameter updates.
 - Need “break the symmetry”
- Small Random Initialization is the standard practice
- Current recommendation for initializing CNNs with RELU: **Why?**
$$w = \text{np.random.randn}(n) * \sqrt{2.0/n}$$
 - “randn”: Gaussian; “n”: the number of inputs for current layer.
- For general NNs, layer-wise pre-training is safe.
- Even safer: start from a pre-trained model

Choice of Activation Functions



Monitor Your Training Curve

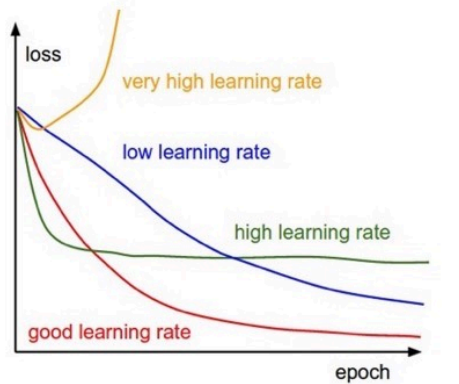
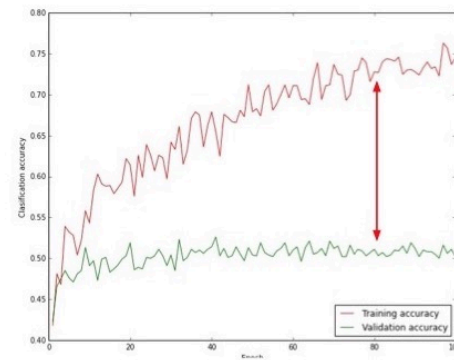


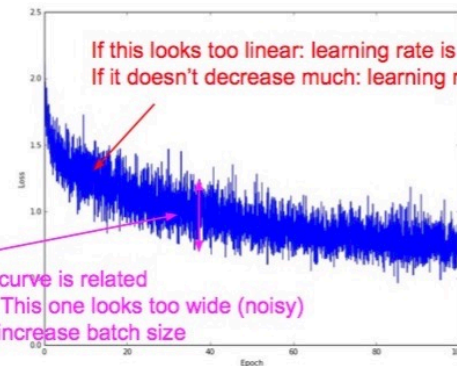
Figure 1



big gap = overfitting
=> increase regularization strength

no gap
=> increase model capacity

Figure 3



the "width" of the curve is related to the batch size. This one looks too wide (noisy)
=> might want to increase batch size

Figure 2

Supervised Learning

Feature Space $\mathcal{X}$

Label Space $\mathcal{Y}$

Goal: Construct a **predictor** $f : \mathcal{X} \rightarrow \mathcal{Y}$ to minimize

$$R(f) \equiv \mathbb{E}_{XY} [\text{loss}(Y, f(X))]$$

Optimal predictor (Bayes Rule) depends on unknown P_{XY} , so instead
learn a good prediction rule from training data $\{(X_i, Y_i)\}_{i=1}^n \stackrel{\text{iid}}{\sim} P_{XY}(\text{unknown})$

$$\max \mathbb{E}_{x,y \sim p(x,y)} [\log p(y|x)]$$

- ML has been largely focused on this ...
 - **But Lots** of other problem settings are coming up:
 - What if we **also** have unlabeled data?
 - What if we **only** have unlabeled data?
 - What if we *have poor-quality labels* (e.g., coarse or potentially mistaken?)
 - What if we have many datasets, but one somehow differing from another?
 - What if we only have one example, or a few per (new) class?
 -

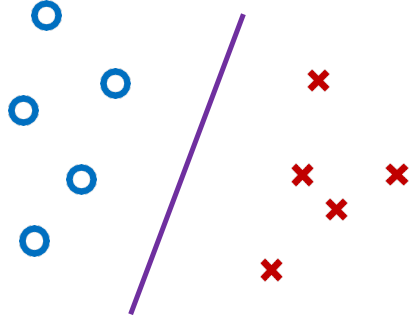
And wait, there are more!

- Transfer Learning
- Semi-supervised learning
- One/Few-shot learning
- Un/Self-Supervised Learning
- Domain adaptation
- Meta-Learning
- Zero-shot learning
- Continual / Lifelong-learning
- Multi-modal learning
- Multi-task learning
- Active learning
- ...

Setting	Source	Target	Shift Type
Semi-supervised	Single labeled	Single unlabeled	None
Domain Adaptation	Single labeled	Single unlabeled	Non-semantic
Domain Generalization	Multiple labeled	Unknown	Non-semantic
Cross-Task Transfer	Single labeled	Single unlabeled	Semantic
Few-Shot Learning	Single labeled	Single few-labeled	Semantic
Un/Self-Supervised	Single unlabeled	Many labeled	Both/Task

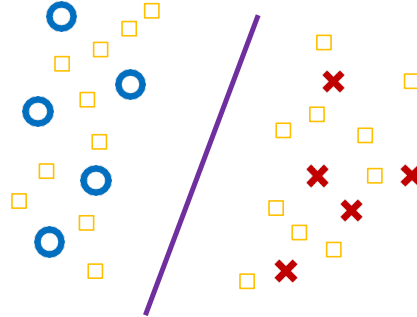
Problem settings in a nutshell

PN learning
(i.e., supervised learning)



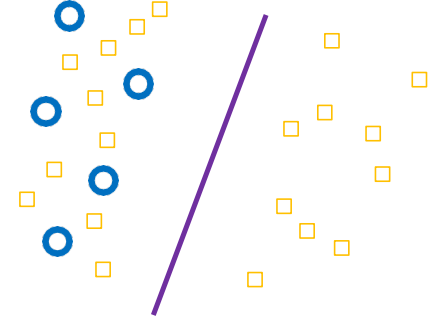
P & N data are
available for training

PNU learning
(i.e., semi-supervised learning)



P, N & U data are
available for training

PU learning



P & U data are
available for training

○ : positive data

✕ : negative data

□ : unlabeled data

Few-Shot Learning

**People are
good at it**



**Human-level concept learning
through probabilistic
program induction**

Brenden M. Lake,^{1*} Ruslan Salakhutdinov,² Joshua B. Tenenbaum³



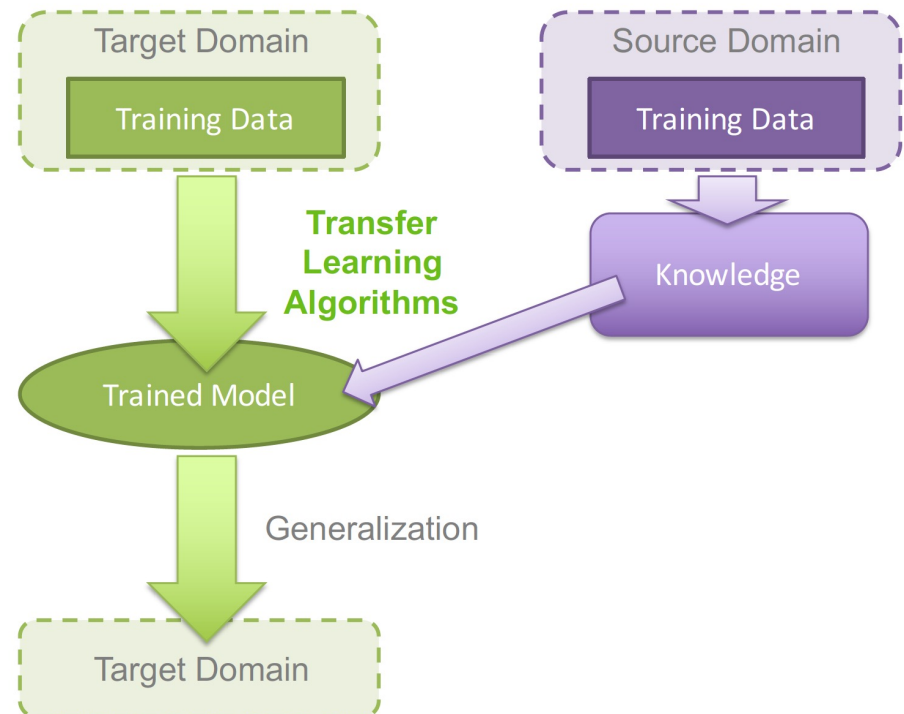
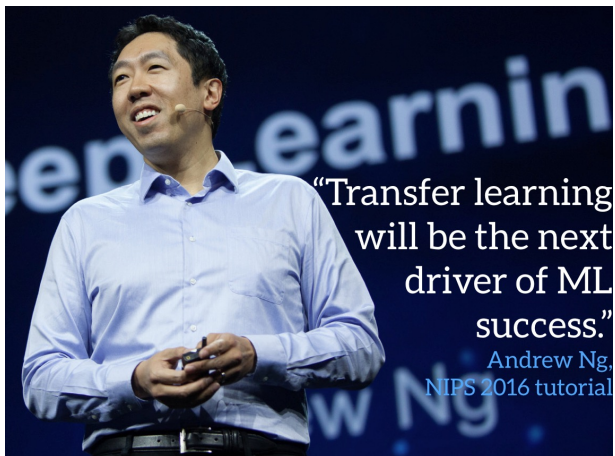
ಕು

**Machines are
getting
better at it**

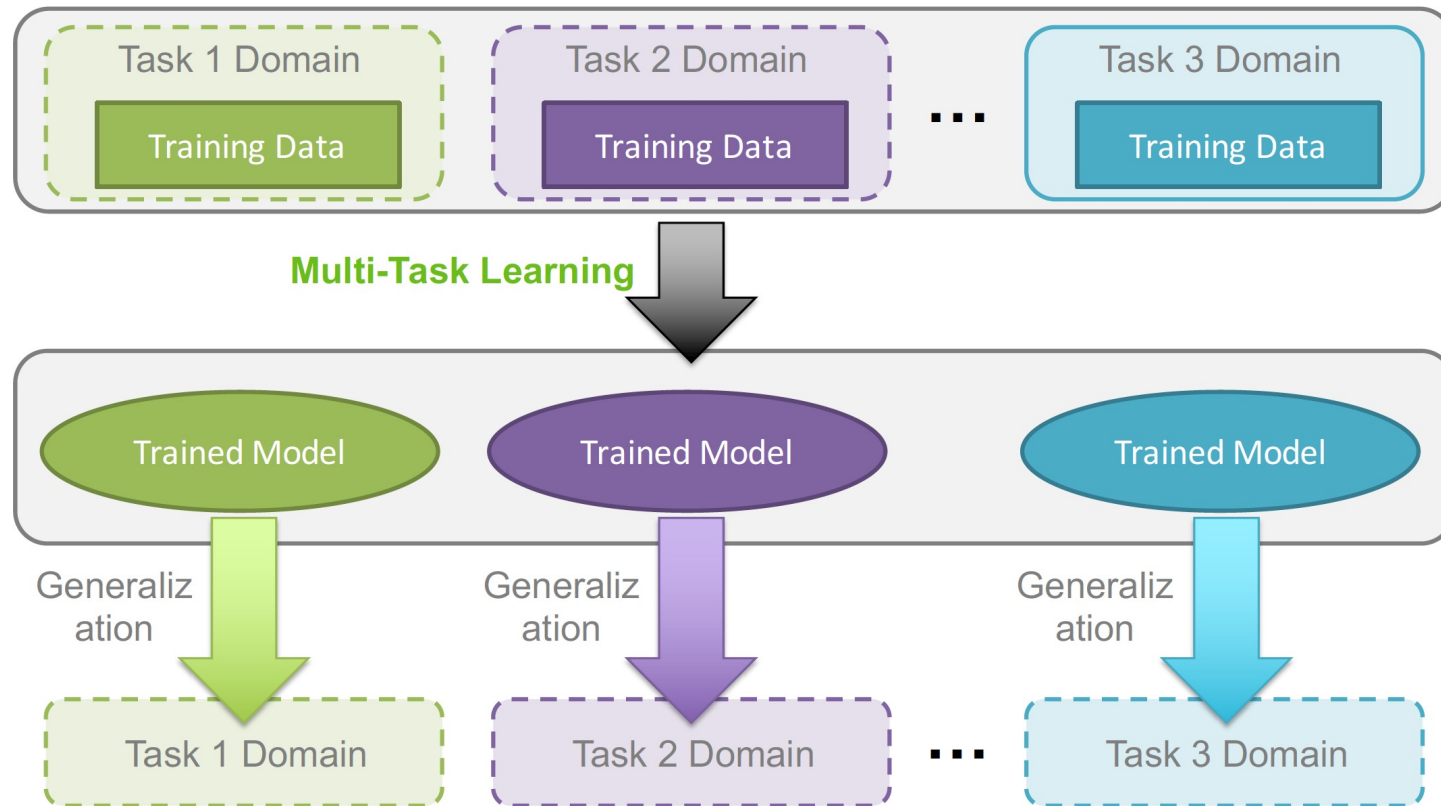
ಅ	ಇ	ಉ	ಎ	ಔ
ಕ	ಖ	ಗ	ಙ	ಚ
ಇ	ಠ	ಡ	ತ	ಥ
ನ	ಯ	ಲ	ವ	ಶ

Transfer Learning

Improve Learning New Task
by Learned Task



Multi-Task Learning



Simple Contrastive Learning (simCLR)

A Simple Framework for Contrastive Learning of Visual Representations

Ting Chen¹ Simon Kornblith¹ Mohammad Norouzi¹ Geoffrey Hinton¹

- **Simple idea:** maximizing the agreement of representations under data transformation, using a contrastive loss in the latent/feature space
- **Super effective:** 10% relative improvement over previous SOTA (cpc v2), outperforms AlexNet with 100X fewer labels

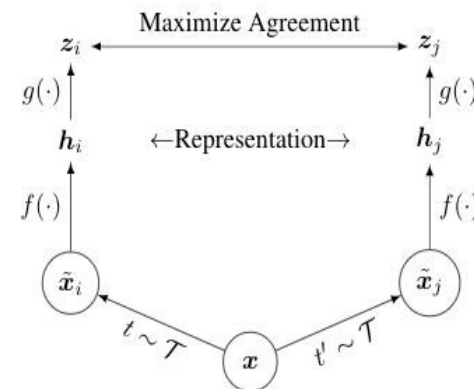
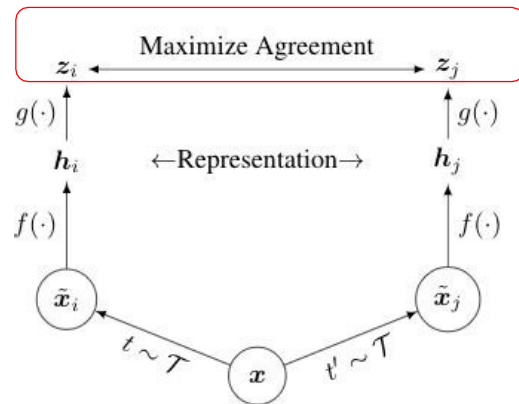


Figure 2. A framework for contrastive representation learning. Two separate stochastic data augmentations $t, t' \sim \mathcal{T}$ are applied to each example to obtain two correlated views. A base encoder network $f(\cdot)$ with a projection head $g(\cdot)$ is trained to maximize agreement in *latent representations* via a contrastive loss.

Simple Contrastive Learning Contrast (simCLR)



Maximize agreement using a contrastive task:

Given $\{x_k\}$ where two different examples x_i and x_j are a positive pair, identify x_j in $\{x_k\}_{k \neq i}$ for x_i .



Loss function:

$$\text{Let } \text{sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u}^\top \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|}$$

$$\ell_{i,j} = -\log \frac{\exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_j)/\tau)}{\sum_{k=1}^{2N} \mathbb{1}_{[k \neq i]} \exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_k)/\tau)}$$